EXPERIMENT - 6

Implement a program for recursion and list in PROLOG

a) Factorial using Recursion

Def: Factorial is a classic example of recursion in Prolog. The base case states that factorial of 0 is 1. For any N > 0, the factorial is computed by recursively calling factorial(N-1, F1) and then multiplying N * F1 using the is operator. Prolog uses backtracking and recursive unification to evaluate the result.

CODE:

```
factorial(0,1).
factorial(N,F) :-
    N > 0,
    N1 is N - 1,
    factorial(N1,F1),
    F is N * F1.
```

Query: ?- factorial(N, F).

Output 1:

```
?- factorial(0,F).
F = 1.
```

Output 2:

```
?- factorial(1,F).
F = 1.
```

Output 3:

```
?- factorial(2,F).
F = 2.
```

Output 4:

```
?- factorial(3,F).
F = 6.
```

Output 5:

?- factorial(4,F).

F = 24.

Output 6:

?- factorial(5,F).

F = 120.

Output 7:

?- factorial(6,F).

F = 720.

Output 8:

?- factorial(7,F).

F = 5040.

Output 9:

?- factorial(8,F).

F = 40320.

Output 10:

?- factorial(9,F).

F = 362880.

b) Fibonacci Number using Recursion

Def: The Fibonacci sequence is defined such that each number is the sum of the two preceding ones. In Prolog, base cases handle fib(0,0) and fib(1,1). For N > 1, the program recursively computes fib(N-1, F1) and fib(N-2, F2), then calculates F is F1 + F2 using arithmetic operators. This demonstrates recursive list and number computation in Prolog.

CODE:

```
fib(0,0).
fib(1,1).
fib(N,F):-
   N > 1,
   N1 is N - 1,
   N2 is N - 2,
   fib(N1,F1),
   fib(N2,F2),
   F is F1 + F2.
```

Query: ?- fib(N, F).

Output 1:

?- fib(0,F).

F = 0.

Output 2:

?- fib(1,F).

F = 1.

Output 3:

?- fib(2,F).

F = 1.

Output 4:

?- fib(3,F).

F = 2.

Output 5:

?- fib(4,F).

F = 3.

Output 6:

?- fib(5,F).

F = 5.

Output 7:

?- fib(6,F).

F = 8.

Output 8:

?- fib(7,F).

F = 13.

Output 9:

?- fib(8,F).

F = 21.

Output 10:

?- fib(9,F).

F = 34.

c) Fibonacci Series using Recursion and List

Def: This program prints the Fibonacci series up to the Nth term using recursion in Prolog. The base case fib_series(0) terminates the recursion. For N > 0, it first recursively prints fib_series(N-1), then computes and writes the Nth Fibonacci number. This uses both recursive predicates and list-style sequential output, demonstrating Prolog's power for generating and displaying mathematical sequences.

CODE:

```
fib(0, 0).
fib(1, 1).
fib(N, F) :-
N > 1,
N1 is N - 1,
N2 is N - 2,
fib(N1, F1),
fib(N2, F2),
F is F1 + F2.
fib_series(0).
fib_series(N) :-
N > 0,
N1 is N - 1,
fib_series(N1),
fib(N, F),
write(F), write(' ').
```

Query: ?- fib_series(N).

Output 1: *(Series up to index 1)*

```
?- fib_series(1).
0 1
```

Output 2: *(Series up to index 2)*

```
?- fib_series(2).
0 1 1
```

Output 3: *(Series up to index 3)*

?- fib_series(3).

0 1 1 2

Output 4: *(Series up to index 4)*

?- fib_series(4).

0 1 1 2 3

Output 5: *(Series up to index 5)*

?- fib_series(5).

0 1 1 2 3 5

Output 6: *(Series up to index 6)*

?- fib_series(6).

0 1 1 2 3 5 8

Output 7: *(Series up to index 7)*

?- fib_series(7).

0 1 1 2 3 5 8 13

Output 8: *(Series up to index 8)*

?- fib_series(8).

0 1 1 2 3 5 8 13 21

Output 9: *(Series up to index 9)*

?- fib_series(9).

0 1 1 2 3 5 8 13 21 34

Output 10: *(Series up to index 10)*

?- fib_series(10).

0 1 1 2 3 5 8 13 21 34 55